

并行程序设计 Lab3: NTT 的 Pthread、CRT 与 OpenMP 优化实验报告

姓名: 强徽东

学号: 2411764

2026 年 5 月 29 日

1 实验背景与目标

本实验选题为 NTT (Number Theoretic Transform, 数论变换) 下的多项式乘法优化。给定两个长度为 n 的多项式 $A(x)$ 和 $B(x)$, 目标是在模数 P 下计算:

$$C(x) = A(x)B(x) \pmod{P}$$

实验按照由浅入深的顺序展开:

1. 实现朴素卷积的 Pthread 并行版本, 作为 baseline。
2. 实现基础 NTT 的 Pthread 优化版本。
3. 实现 CRT 合并的 NTT 版本, 包括三模数和四模数版本。
4. 实现 CRT 的 Pthread 优化版本。
5. 进一步使用 OpenMP 实现自动调度优化。
6. 在 OpenMP 版本基础上尝试使用 `#pragma omp simd` 自动向量化。

实验中特别注意平台约束: 每个任务只申请 1 个 CPU (8 核), 因此多线程版本均控制总工作线程数不超过 8, 并尽量采用“最多 7 个子线程 + 主线程参与计算”的模式。

2 朴素卷积的 Pthread 优化

2.1 串行朴素算法

朴素多项式乘法的直接写法为:

```

for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        ab[i + j] = (ab[i + j] + a[i] * b[j]) % p;
    }
}

```

该算法复杂度为 $O(n^2)$ 。如果直接并行外层循环，不同线程可能同时写入同一个 $ab[i+j]$ ，需要加锁或使用原子操作，会引入较大的同步开销。

2.2 按输出系数并行

本实验采用按输出系数 k 分块的方式：

$$ab[k] = \sum_i a[i]b[k-i] \pmod{P}$$

其中 i 的范围满足：

$$0 \leq i < n, \quad 0 \leq k-i < n$$

因此：

$$i \in [\max(0, k-n+1), \min(n-1, k)]$$

每个线程负责一段连续的 k ，不同线程写入不同的 $ab[k]$ ，因此不存在数据竞争，也不需要互斥锁。

```

for (int k = task->l; k < task->r; ++k) {
    int begin = std::max(0, k - n + 1);
    int end = std::min(n - 1, k);

    u128 sum = 0;
    for (int i = begin; i <= end; ++i) {
        sum += (u128)a[i] * b[k - i];
        if ((i & 255) == 255) sum %= mod;
    }
    ab[k] = (u64)(sum % mod);
}

```

该版本作为 baseline，能够验证 Pthread 编程框架和任务划分方式，但由于复杂度仍为 $O(n^2)$ ，在 $n = 131072$ 时运行时间较长。

2.3 线程数量设计

根据实验要求，线程总数控制在 8 以内。实际实现中：

```
int worker_threads = threads - 1;
```

只创建 threads-1 个子线程，主线程也执行一段计算任务：

```
plain_worker(&tasks[worker_threads]);
```

这样可以避免额外创建第 8 个子线程，同时让主线程承担计算任务。

3 基础 NTT 的 Pthread 优化

3.1 NTT 基本流程

NTT 将多项式卷积转化为点值乘法，整体流程如下：

$$A \xrightarrow{\text{NTT}} \hat{A}, \quad B \xrightarrow{\text{NTT}} \hat{B}$$

$$\hat{C}[i] = \hat{A}[i]\hat{B}[i] \pmod{P}$$

$$C \xleftarrow{\text{INTT}} \hat{C}$$

复杂度由朴素卷积的 $O(n^2)$ 降为 $O(n \log n)$ 。

3.2 蝶形循环并行

基础 NTT 的核心循环为：

```
for (int len = 2; len <= n; len <<= 1) {
    int half = len >> 1;
    for (int base = 0; base < n; base += len) {
        for (int k = 0; k < half; ++k) {
            // butterfly
        }
    }
}
```

同一层内不同蝶形块之间互不依赖，可以并行；但不同层之间存在依赖，因此每一层结束后需要同步。本实验中使用 `pthread_barrier_wait` 保证层间同步。

在前中期，蝶形块数量较多，按照 block 分配；在后期 block 数量少于线程数时，改为按 butterfly 数量划分，避免只有少数线程工作。

4 CRT 实现任意模数 NTT

4.1 CRT 思想

当目标模数 P 过大，或者不是 NTT 友好模数时，可以选择若干个 NTT 友好小模数 $p_1, p_2, \dots, p_m$ 。先分别在小模数下计算卷积：

$$C_i(x) = A(x)B(x) \pmod{p_i}$$

对于每个结果系数 $c[k]$ ，有：

$$c[k] \equiv C_i[k] \pmod{p_i}$$

使用中国剩余定理恢复 $c[k] \pmod{M}$ ，其中：

$$M = \prod_{i=1}^m p_i$$

最后再对目标模数 P 取模：

$$ans[k] = c[k] \bmod P$$

CRT 正确恢复的前提是：

$$n(P-1)^2 < M$$

4.2 三模数版本

讲义中介绍的是三模数合并。本实验实现的三模数为：

$$998244353, \quad 1004535809, \quad 469762049$$

三者乘积约为：

$$4.71 \times 10^{26}$$

该版本符合讲义模板，但对于特别大的目标模数，覆盖范围可能不足。

4.3 四模数版本

为了支持实验中出现的大模数，例如：

$$P = 263882790666241$$

以及测试中出现的：

$$P = 1337006139375617$$

进一步实现了四模数版本：

$$998244353, \quad 1004535809, \quad 469762049, \quad 1224736769$$

其中四个模数均为 NTT 友好模数，原根均可使用 3。四模数乘积约为：

$$5.77 \times 10^{35}$$

能够覆盖 $n = 131072$ 、 $P = 1337006139375617$ 时的最坏卷积系数上界。

5 CRT 的 Pthread 优化

CRT 版本天然具有粗粒度并行性。不同小模数下的 NTT 计算相互独立，因此可以并行执行。

四模数 Pthread 版本采用如下结构：

$$4 \text{ 个小模数任务} = 3 \text{ 个子线程} + 1 \text{ 个主线程}$$

即：

```
const int worker_threads = CRT_CNT - 1;

for (int i = 0; i < worker_threads; ++i) {
    pthread_create(&tids[i], nullptr, mod_worker, &tasks[i]);
}

run_mod_worker(&tasks[worker_threads]);
```

小模数 NTT 全部完成后，再进行 CRT 合并。CRT 合并是按系数独立进行的，因此也可以按输出区间并行。

6 OpenMP 自动调度优化

6.1 内部 NTT 并行版本

在 OpenMP 版本中，首先实现了 NTT 内部并行：四个模数串行处理，但每个小模数内部的 NTT 蝶形循环使用 OpenMP 并行。

对于每层 NTT：

- 当 block 数量较多时，使用 `schedule(static)`。
- 当 block 数量较少时，拆分 butterfly，避免线程空闲。

典型代码如下：

```
#pragma omp parallel for schedule(static)
for (int block = 0; block < blocks; ++block) {
    // butterfly block
}
```

CRT 合并阶段使用：

```
#pragma omp parallel for schedule(guided, 256)
for (int i = 0; i < result_len; ++i) {
    // CRT combine
}
```

6.2 模数并行版本

随后实现了另一种 OpenMP 优化方式：模数并行。外层四个小模数任务并行，每个小模数内部 NTT 保持串行，避免嵌套并行。

```
int mod_threads = std::min(get_omp_threads(), CRT_CNT);

#pragma omp parallel for schedule(static) num_threads(mod_threads)
for (int i = 0; i < CRT_CNT; ++i) {
    multiply_mod_ntt(a, b, residues[i], n, CRT_MODS[i]);
}
```

这种方式最多使用 4 个线程，线程管理开销较小，但无法完全利用 8 核。对于 $n = 131072$ 的大规模 NTT，内部 NTT 并行通常更容易充分利用核心；而模数并行作为对照实验，可以用于分析不同粒度并行策略的性能差异。

7 OpenMP SIMD 自动向量化

在模数并行 OpenMP 版本基础上，又实现了 OpenMP SIMD 自动向量化版本。由于 NTT 内层原始写法中存在：

$$w \leftarrow w \cdot wn$$

这种循环依赖，不利于编译器向量化。因此先预计算旋转因子数组：

```
std::vector<u64> roots(half);
roots[0] = 1;
for (int k = 1; k < half; ++k) {
    roots[k] = mul_mod_ntt(roots[k - 1], wn, mod);
}
```

之后在蝶形循环中使用：

```
#pragma omp simd
for (int k = 0; k < half; ++k) {
    u64 x = f[base + k];
    u64 y = mul_mod_ntt(f[base + k + half], roots[k], mod);
    f[base + k] = add_mod(x, y, mod);
    f[base + k + half] = sub_mod(x, y, mod);
}
```

其中 `mul_mod_ntt` 使用 64 位乘法：

```
static inline u64 mul_mod_ntt(u64 x, u64 y, u64 mod) {
    return x * y % mod;
}
```

由于 CRT 小模数均小于 1.3×10^9 ，乘积不超过 1.6×10^{18} ，仍在 `uint64_t` 范围内，因此该写法在小模数 NTT 内部是安全的。对于目标大模数下的通用乘法，仍应使用 `__uint128_t`。

8 实验结果

以下为部分已测试结果。时间单位以程序输出为准。

表 1: 朴素 Pthread baseline 测试结果

样例	n	模数 P	时间 / ms
0	4	7340033	0.598961
1	131072	7340033	7434.47

表 2: 四模数 CRT 版本调试过程中的测试结果

样例	n	模数 P	结果
0	4	7340033	正确
1	131072	7340033	正确
2	131072	104857601	正确
3	131072	469762049	正确
4	131072	1337006139375617	更换第四模数后正确

9 问题与分析

9.1 三模数覆盖范围不足

三模数 CRT 虽然符合讲义模板，但当目标模数较大时，可能无法覆盖真实卷积系数上界：

$$n(P-1)^2$$

因此在大模数测试中，需要使用四模数 CRT。实验中最初使用第四模数 754974721 时，大模数样例出现错误；将第四模数替换为 1224736769 后，四模数乘积增大，最终结果正确。

9.2 线程粒度选择

CRT 可以在模数层并行，也可以在 NTT 内部并行。模数并行粒度较粗，最多只有 4 个任务；NTT 内部并行粒度更细，能够更充分使用 8 核。因此对于大规模 $n = 131072$ ，内部 NTT 并行通常更合适。

9.3 SIMD 自动向量化限制

`#pragma omp simd` 只是向编译器提供向量化提示，并不保证一定生成 SIMD 指令。由于模乘中存在取模运算，编译器能否完全向量化取决于平台、编译器和优化参数。建议使用：

```
g++ -std=c++17 -O3 -fopenmp -march=native main.cc -o main
```

10 总结

本实验围绕 NTT 多项式乘法实现了多层次优化。朴素 Pthread 版本验证了基本线程划分方法，但复杂度较高；基础 NTT 将复杂度降至 $O(n \log n)$ ；CRT 版本解决了大

模数或非 NTT 友好模数的问题；Pthread 和 OpenMP 版本分别从手动线程控制和自动调度两个角度进行了并行优化；最后使用 OpenMP SIMD 尝试进一步利用向量化能力。

从实现过程看，CRT 模数数量和模数选择会直接影响大模数样例的正确性；线程数量需要结合平台核心数控制；OpenMP 的 `static`、`guided` 和 `simd` 可以分别用于规则循环、负载不均循环和内层向量化尝试。